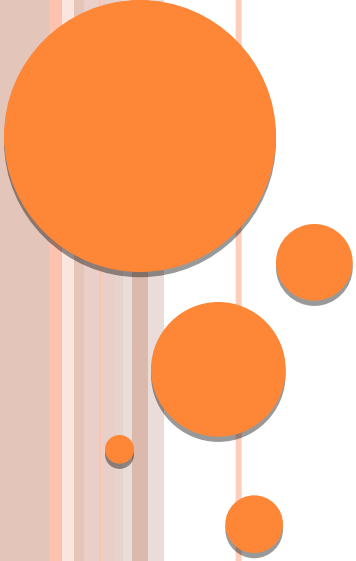




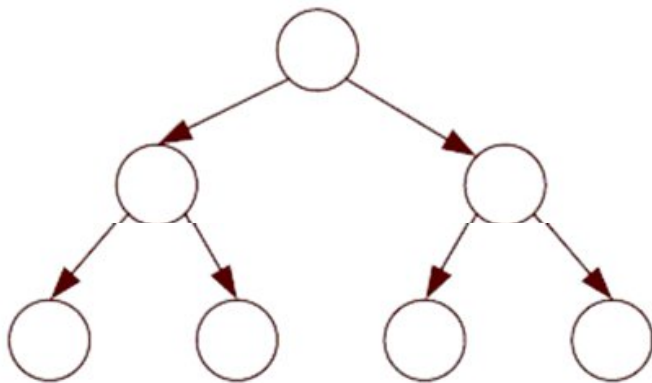
PROGRAMMATION DYNAMIQUE

DA IODAA – Tools - AgroParisTech

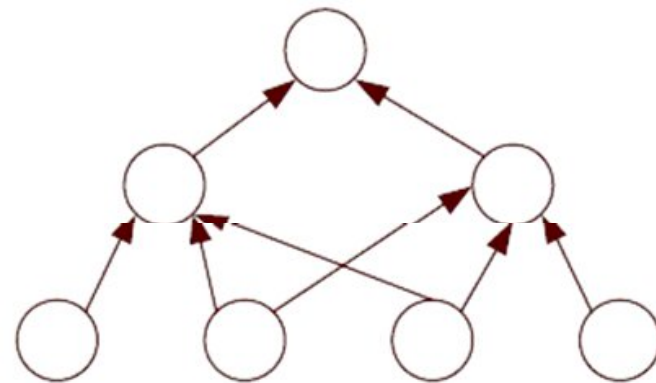
Christine MARTIN

PROGRAMMATION DYNAMIQUE

- Paradigme de programmation
 - Adaptation de la méthode diviser pour régner
- Principe de Bellman (années 50)
 - Solution du problème obtenu à partir des solutions de sous-problèmes



Approche diviser pour régner



Approche programmation
dynamique

PROGRAMMATION DYNAMIQUE

SUITES DE FIBONACCI

- On souhaite calculer les n premiers termes de la suite définie par :
 - $F(0) = 1; F(1) = 1;$
 - $F(n) = F(n-1) + F(n-2);$
- Algorithme classique :

```
8 def fibo(n) :  
9     print "calcul pour n = ", n  
10    if n<=1 :  
11        return 1  
12    else :  
13        return fibo(n-1) + fibo(n-2)  
14  
15 print fibo(4)  
16
```

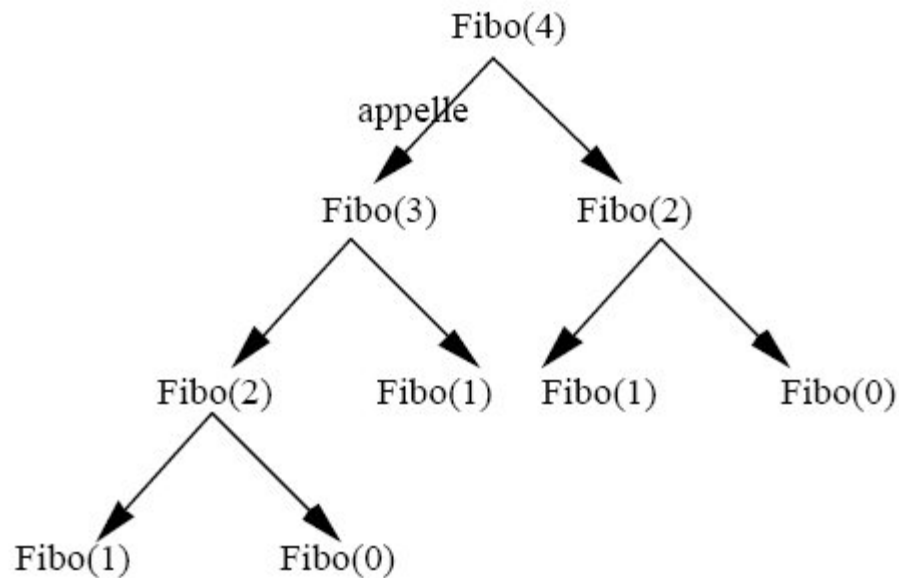


```
calcul pour n = 4  
calcul pour n = 3  
calcul pour n = 2  
calcul pour n = 1  
calcul pour n = 0  
calcul pour n = 1  
calcul pour n = 2  
calcul pour n = 1  
calcul pour n = 0  
5
```

- Pb : multiplicité de calcul d'un même nombre !

PROGRAMMATION DYNAMIQUE

SUITES DE FIBONACCI



Appels successifs pour $n=4$

PROGRAMMATION DYNAMIQUE

SUITES DE FIBONACCI

- Idée :
 - Sauvegarder les résultats obtenus
 - Vérifier si un résultat a déjà été obtenus avant de le calculer
 - Ici on peut stocker les résultats intermédiaires dans une liste
- Nouvel algorithme :

```
18 def fibo_bis(n):
19     vals = [1,1]
20     i = 2
21     while i<=n :
22         vals.append(vals[-1] + vals[-2])
23         i += 1
24     return vals[n]
25
```

PROGRAMMATION DYNAMIQUE

MÉTHODE

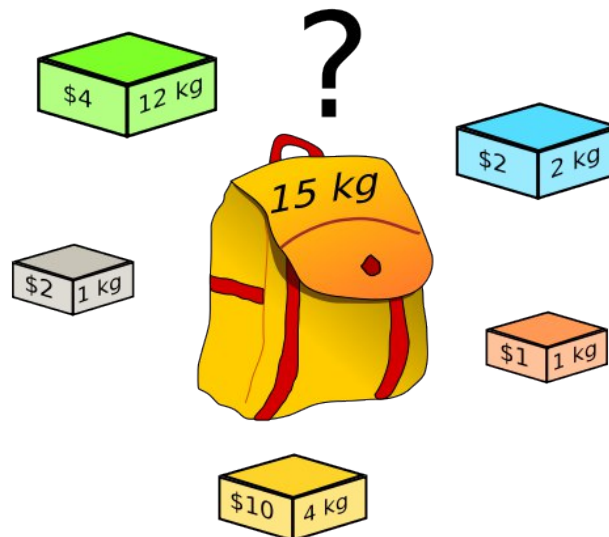
1. Obtention de l'équation récursive
 - Lien entre la solution d'un problème et celle de sous-problèmes
2. Initialisation de la table
 - Conditions initiales de l'équation obtenue en 1
3. Remplissage de la table
 - À partir de l'équation obtenue en 1
4. Lecture de la solution
 - Pas toujours de solution directe
 - Un parcours dans la table est souvent nécessaire !

PROGRAMMATION DYNAMIQUE

LE PROBLÈME DU SAC À DOS

○ Problème

- On a un sac à dos de capacité maximale **W** (poids) et **n** objets (**x_i** , **w_i**) de valeurs et de poids diverses.
- On souhaite remplir le sac de façon à cumuler un maximum de valeurs sans dépasser sa capacité.
- Quel est le bénéfice maximum que l'on peut faire ?



PROGRAMMATION DYNAMIQUE

LE PROBLÈME DU SAC À DOS

- Construction de la solution optimale du problème à i variables à partir des sous-problèmes associés
 - programmation dynamique.
 - ici la capacité et le poids !
 - optimiser le gain
- On notera $S(i,c)$ le problème réduit à i objets et à la contenance c .
- Comment déduire la solution d'un problème à i objets et une capacité c de ses sous-problèmes et quels sont-ils ?

PROGRAMMATION DYNAMIQUE

LE PROBLÈME DU SAC À DOS

- L'idée est la suivante :
 - Le problème à zéro objet ($S(0,*)$) a une solution optimale de valeur nulle.
 - Pour tout valeur de capacité
 - Étant donné un nouvel objet i et une contenance c , les solutions optimales de $S(i,c)$ sont soit :
 - les solutions optimales du problème à $i-1$ objets avec la même contenance c ($S(i-1,c)$)
 - → On n'a pas la capacité pour rajouter l'objet i
 - les solutions optimales du problème à $i-1$ objets avec la contenance $c - w_i$ ($S(i-1, c - w_i)$) auquel il faut rajouter le gain apporté par l'objet i
 - → On a la possibilité de rajouter i étant donnée c
 - On conserve la meilleure !

PROGRAMMATION DYNAMIQUE

LE PROBLÈME DU SAC À DOS

- Dans les sous-problèmes :
 - Le nombre d'objets et la capacité à considérer varient !
- On construit alors une table $T[i,c]$ contenant la valeur des solutions optimales de tout problème $S(i,c)$.
- On en déduit ensuite la valeur recherchée :
 - Lecture directe dans la table
 - Où ?
- On en déduit enfin la liste des objets retenus :
 - Un parcours de la table est nécessaire !

PROGRAMMATION DYNAMIQUE

LE PROBLÈME DU SAC À DOS

○ Algorithme :

pour c de 0 à W **faire**

$T[0,c] := 0$

fin pour

pour i de 1 à n **faire**

pour c de 0 à W **faire**

si $c \geq w[i]$ **alors**

$T[i,c] := \max(T[i-1,c], T[i-1, c-w[i]] + x[i])$

sinon $T[i,c] := T[i-1,c]$

fin si

fin pour

fin pour

PROGRAMMATION DYNAMIQUE

LE PROBLÈME DU SAC À DOS

- Exemple :

contenance maximale = 12

	1	2	3	4	5	6	7	8
objets :	A	B	C	D	E	F	G	H
poids :	2	3	5	2	4	6	3	1
valeurs :	5	8	14	6	13	17	10	4

- Construisez la table de programmation dynamique associée.
- Déduisez-en la solution au problème posé.

PROGRAMMATION DYNAMIQUE

LE PROBLÈME DU SAC À DOS (CORRECTION)

i\c	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	5	5	5	5	5	5	5	5	5	5	5
2	0												
3	0												
4	0												
5	0												
6	0												
7	0												
8	0												

$$T[1,2] := \max(T[0,2], T[0,0] + w[1])$$
$$T[1,2] := \max(0, 5)$$

	1	2	3	4	5	6	7	8
objets :	A	B	C	D	E	F	G	H
poids :	2	3	5	2	4	6	3	1
valeurs :	5	8	14	6	13	17	10	4

PROGRAM
LE

		1	2	3	4	5	6	7	8
objets :		A	B	C	D	E	F	G	H
poids :		2	3	5	2	4	6	3	1
valeurs :		5	8	14	6	13	17	10	4

i\c	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	5	5	5	5	5	5	5	5	5	5	5
2	0	0	5	8	8	13	13	13	13	13	13	13	13
3	0	0	5	8	8	14	14	19	22	22	27	27	27
4	0	0	6	8	11	14	14	20	22	25	28	28	33
5	0	0	6	8	13	14	19	21	24	27	28	33	35
6	0	0	6	8	13	14	19	21	24	27	30	33	36
7	0	0	6	10	13	16	19	23	24	29	31	34	37
8	0	4	6	10	14	17	20	23	27	29	33	35	38

PROGRAMMATION DYNAMIQUE

LE PROBLÈME DU SAC À DOS

- **Récupérer la liste d'objets**
- Une fois la table construite, il « suffit »
 - de démarrer de la case de $T[n, W]$
 - et de déduire l'état des objets en remontant jusqu'à une case $T[0, *]$.
- Comment ?
 - Analysant le contenu de la table et les valeurs identiques
 - Si des valeurs successives sont identiques
 - L'optimum est obtenu avec
 - une capacité plus petite
 - et /ou moins d'objets

PROGRAMMATION DYNAMIQUE

LE PROBLÈME DU SAC À DOS

- **Récupérer la liste d'objets**
- Comment ?
 - Analysant le contenu de la table et les valeurs identiques
 - Je suis sur une case
 - Si j'ai la même valeur à ma gauche, qu'est-ce que cela signifie ?
 - Si j'ai la même valeur au dessus, qu'est-ce que cela signifie ?
 -

PROGRAMMATION DYNAMIQUE

LE PROBLÈME DU SAC À DOS

- Une fois la table construite, il suffit de démarrer de la case de $T[n, W]$ et de déduire l'état des objets en remontant jusqu'à une case $T[0, *]$.
- **Récupérer la liste d'objets**
 - Tout d'abord, on récupère le poids minimal nécessaire pour faire le bénéfice optimal :
TANT QUE $T[i, j] \neq T[i, j-1]$
 $j = j-1$
 - Et puis, de là, on récupère les objets :
TANT QUE $j > 0$
 TANT QUE $i > 0$ ET $T[i, j] \neq T[i-1, j]$
 $i = i-1$
 $j = j - \text{PoidsObjet}[i]$
 SI $j \geq 0$
 Ajoute-objet ($\text{Objet}[i]$)
 $i = i-1$

PROGRAM LE

		1	2	3	4	5	6	7	8
objets :	A	B	C	D	E	F	G	H	
poids :	2	3	5	2	4	6	3	1	
valeurs :	5	8	14	6	13	17	10	4	

i\c	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	5	5	5	5	5	5	5	5	5	5	5
2	0	0	5	8	8	13	13	13	13	13	13	13	13
3	0	0	5	8	8	14	14	19	22	22	27	27	27
4	0	0	6	8	11	14	14	20	22	25	28	28	33
5	0	0	6	8	13	14	19	21	24	27	28	33	35
6	0	0	6	8	13	14	19	21	24	27	30	33	36
7	0	0	6	10	13	16	19	23	24	29	31	34	37
8	0	4	6	10	14	17	20	23	27	29	33	35	38

SAC À DOS (CORRECTION)

	1	2	3	4	5	6	7	8
objets :	A	B	C	D	E	F	G	H
poids :	2	3	5	2	4	6	3	1
valeurs :	5	8	14	6	13	17	10	4

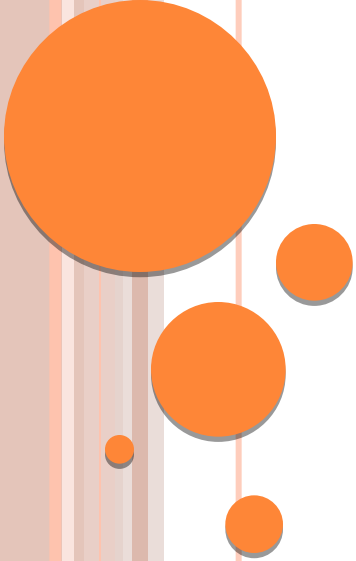
i\c	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	5	5	5	5	5	5	5	5	5	5	5
2	0	0	5	8	8	13	13	13	13	13	13	13	13
3	0	0	5	8	8	14	14	19	22	22	27	27	27
4	0	0	6	8	11	14	14	20	22	25	28	28	33
5	0	0	6	8	13	14	19	21	24	27	28	33	35
6	0	0	6	8	13	14	19	21	24	27	30	33	36
7	0	0	6	10	13	16	19	23	24	29	31	34	37
8	0	4	6	10	14	17	20	23	27	29	33	35	38

La solution est donc : **A, D, E, G, H** pour une valeur de **38**

PROGRAMMATION DYNAMIQUE

QUAND ET COMMENT L'UTILISER ?

- Outil général
- Mais pas de règle !
- Obtenir une formulation récursive
 - Solution optimale à partir de sous-solutions optimales
 - Principe de Bellman
- Analyse de l'implémentation classique
 - Si calcul multiple d'un même sous-problème
 - Alors une approche par programmation dynamique sera bénéfique
- Définition de la table de stockage (dim 1, 2, 3, ...)
 - Une case = un sous-problème
 - Le nombre de sous-problèmes peut être très grand !



PROGRAMMATION DYNAMIQUE BIOINFORMATIQUE

DA IODAA – AgroParisTech

Christine MARTIN

LA BIO-INFORMATIQUE

- De Multiples définitions
 - Domaine d'interface
 - Débuts dans les années 70

Résoudre des problèmes issus de la biologie par des moyens informatiques

Biologie Moléculaire

- 1865 Théorie de l'hérédité (Mendel)
- 1944 ADN support de l'information génétique (Avery)
- 1951 première protéine séquencée : insuline (Sanger)
- 1953 structure en double hélice de l'ADN (Watson-Crick)
- 1961 Code génétique (Nirenberg et Matthaei)
- 1977 Séquençage d'ADN
- 1984 Invention de la PCR
- 1990 Programme international de séquençage du génome humain
- 1996 Génome de la levure
- 2000 Brouillon du génome humain

Bio-informatique

- 1967 Arbre phylogénétique
- 1970 Comparaison de séquences
- 1974 Prédiction de structures de protéines
- 1978 Premières banques de données
- 1980 Alignement local
- 1990 Logiciel Blast
- 1991 Prédiction de gènes

- 1978 : séquence du phage phiX174 (premier génome à ADN, 5386 bp)
- Bacteries
 - 1995 : Haemophilus Influenzae
 - 1996 : Bacillus Subtilis
 - 1996 : Escherichia Coli
- Eucaryotes
 - 1990 : programme international de séquençage du Génome Humain
 - 1996 : levure (premier eucaryote)
 - 1998 : Caenorhabditis Elegans (pluri-cellulaire)
 - 2000 : Arabidopsis Thaliana (premier génome de plante)
 - 2000 : brouillon du génome humain
- Aujourd'hui : plusieurs centaines de génomes publiés
 - souris, drosophile, rat, zebra fish, maïs, poulet, chimpanzé, etc.
- <http://www.genomesonline.org/>

UNE COURSE VERS LES DONNÉES

- Organisation rapide de bases de données et d'échanges entre les différents continents

LES BASES DE DONNÉES NUCLÉIQUES

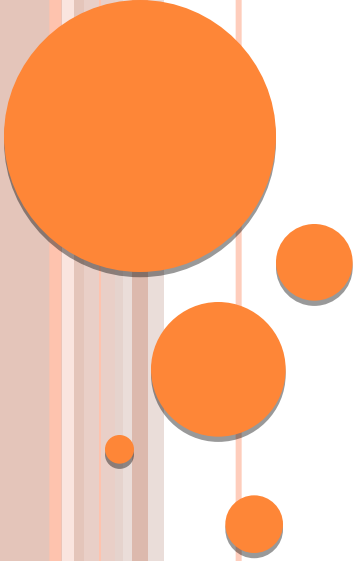
- International Nucleotide Sequence Database Collaboration
- Trois partenaires
 - EMBL (Europe)
 - Genbank (Etats-Unis)
 - DDBJ (Japon)
- Contributeurs: organismes de recherche, programmes de séquençage
- Tous types de séquences
 - chromosome, contigs, ARN messagers, EST, . . .

LES BASES DE DONNÉES PROTÉIQUES

- Swissprot : banques de protéines séquencées
- TrEMBL : banques de protéines prédites à partir d'ARN messagers
- PDB : Protein Data Bank - banque de structures de protéines résolues expérimentalement
cristallographie, résonance magnétique nucléaire
- Lien entre la séquence et la structure dans PDB, toutes les protéines avec plus de 25 % d'identité partagent la même structure



Comparer des séquences



ALIGNEMENTS DE SÉQUENCES

SIMILARITÉ ENTRE SÉQUENCES ET ALIGNEMENTS

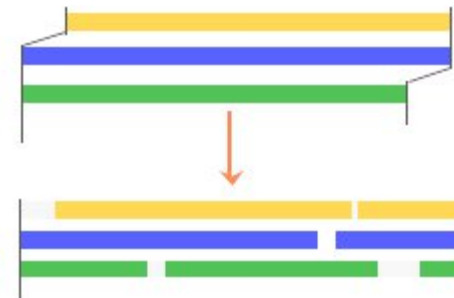
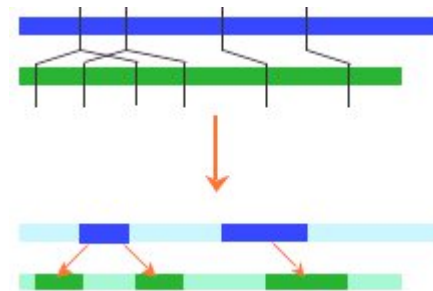
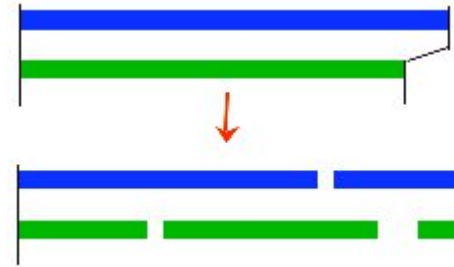
- Indiquer :
 - Prédire un gène
 - Une fonction biologique proche
 - Une structure tridimensionnelle semblable
 - Une origine commune (ancêtre commun)
 - Homologues
 - ...
- On aligne puis on décide si le lien est significatif ou non.

EXEMPLE : L'INSULINE

Éléphant	FVNQHLCGSHLVEALYLVCGERGFFYTPKTGIVEQCCTGVCSLYQLENYCN
Hamster	FVNQHLCGSHLVEALYLVCGERGFFYTPKSGIVDQCCTSI
Éléphant	FVNQHLCGSHLVEALYLVCGERGFFYTPKTGIVEQCCTGVCSLYQLENYCN
Baleine	FVNQHLCGSHLVEALYLVCGERGFFYTPKAGIVEQCCASTCSLYQLENYCN
Éléphant	FVNQHLCGSHLVEALYLVCGERGFFYTPKTGIVEQCCTGVCSLYQLENYCN
Alligator	AANQRLCGSHLVDALYLVCGERGFFYSPKGGIVEQCCHNTCSLYQLENYCN

ALIGNEMENTS DE SÉQUENCES

- Alignement global
 - Mesure du degré de similitude
 - Sur l'ensemble de la séquence
- Alignement local
 - Régions isolées
 - Haut degré de similitude
 - Références, bases
- Alignement multiple
 - Plusieurs séquences entières
 - Relations entre séquences



SIMILARITÉ ENTRE SÉQUENCES ET ALIGNEMENTS

○ Problèmes :

- Définir le type d'alignements souhaités
- Déterminer un système de scores
- Trouver des algorithmes d'alignements optimaux

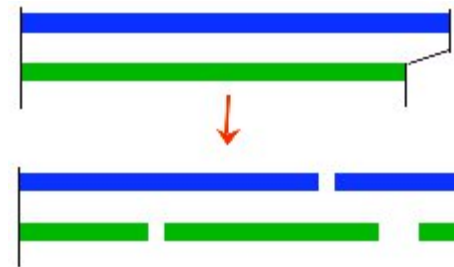
ALIGNEMENT DE SÉQUENCES

MÉTHODE GLOBALE

- Mise en correspondance de deux séquences (ADN ou protéines) dans leur totalité

R	D	I	S	L	V	-	-	-	K	N	A	G	I
R	N	I	-	L	V	S	D	A	K	N	V	G	I

- Identité (match)
- 3 évènements mutationnels élémentaires
 - substitution (mismatch)
 - insertion } Indel (gap)
 - délétion }



SCORES ET MATRICES DE SUBSTITUTION

- Score d'une opération
 - substitution : score de similarité
 - in-del : pénalité
 - Dérivés d'un modèle probabiliste :
 - On favorise la conservation par rapport à l'aléatoire (score positif)
 - On pénalise la substitution non conservatrices (score négatif)
- Le score de l'alignement est la somme des scores élémentaires
 - Hypothèse d'indépendance
 - Les mutations à différents endroits sont apparues indépendamment.

ALIGNEMENT DE SÉQUENCES

- 2 séquences → plusieurs alignements possibles

R	D	I	S	L	V	-	-	-	K	N	A	G	I
R	N	I	-	L	V	S	D	A	K	N	V	G	I

R	D	I	-	-	S	L	V	K	N	A	-	-	-	G	I
R	N	I	L	V	S	-	-	-	D	A	K	N	V	G	I

R	D	I	-	-	S	L	V	K	N	A	G	I
R	N	I	L	V	S	D	A	K	N	V	G	I

- matrices de substitutions :

Mismatch	Match	Indel
DN : 1	G, N : 6	-5
AV, LD : 0	R, K : 5	
	A, I, L, S, V : 4	

Quels sont les scores des alignements précédents ?

ALIGNEMENT DE SÉQUENCES

- 2 séquences → plusieurs alignements possibles

R	D	I	S	L	V	-	-	-	K	N	A	G	I
R	N	I	-	L	V	S	D	A	K	N	V	G	I

R	D	I	-	-	S	L	V	K	N	A	-	-	-	G	I
R	N	I	L	V	S	-	-	-	D	A	K	N	V	G	I

R	D	I	-	-	S	L	V	K	N	A	G	I	
R	N	I	L	V	S	D	A	K	N	V	G	I	

Scores : 19, -11 et 25 respectivement

- Bon/mauvais alignement ? Score maximal

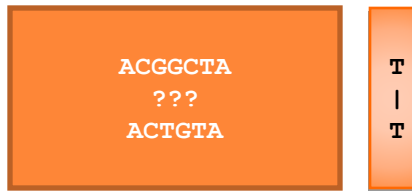
ALIGNEMENT GLOBAL

NEEDLEMAN & WUNSCH (1970)

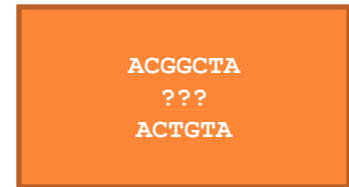
- But : Évaluer une ressemblance globale entre deux séquences
- Données
 - deux séquences (nucléotides ou acides aminés),
 - des scores de similarité et des pénalités.
- Problème
 - Quel est l'alignement de score maximal ?

ALGORITHME

- Séquences : ACGGCTAT et ACTGTAT, Scores : match = 2, mismatch = -1 et in-del = -2.
- Que peut-il se passer pour la dernière opération?
 - Substitution de T en T $\rightarrow +2$

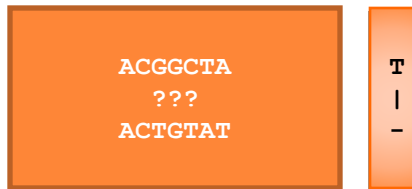


Score de

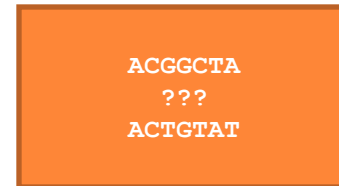


+ 2

- Délétion de T $\rightarrow -2$

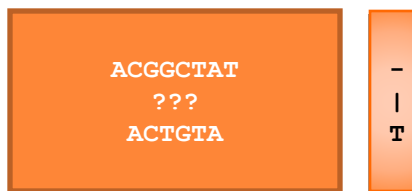


Score de

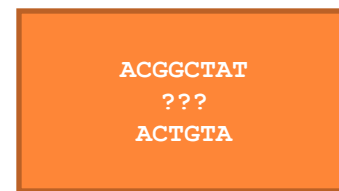


- 2

- Insertion de T $\rightarrow -2$



Score de



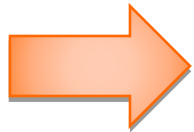
- 2

ALGORITHME

- $\text{Sim}(i, j)$: score optimal entre $U[:i]$ et $V[:j]$
- Formule de récurrence :
 - $\text{Sim}(0, 0) = 0$
 - $\text{Sim}(0, j) = \text{Sim}(0, j - 1) + \text{Ins}(V[j])$
 - $\text{Sim}(i, 0) = \text{Sim}(i - 1, 0) + \text{Del}(U[i])$
 - $\text{Sim}(i, j) = \max \left\{ \begin{array}{l} \text{Sim}(i - 1, j - 1) + \text{Sub}(U[i], V[j]) \\ \text{Sim}(i - 1, j) + \text{Del}(U[i]) \\ \text{Sim}(i, j - 1) + \text{Ins}(V[j]) \end{array} \right.$

ALGORITHME

- Que se passe-t-il si on implémente ces formules de manière récursive ?
 - On calcule de nombreuses fois le même élément !
 - Pas optimal



approche par programmation dynamique

PROGRAMMATION DYNAMIQUE

- Formulation d'un problème sous forme récursive le problème est réduit à plusieurs instances plus petites, elles-mêmes résolues par décomposition.
- Stockage des résultats intermédiaires dans une table
- Extraction de la solution optimale à partir de la table
- Ici :

Calculs intermédiaires

=

Scores d'alignements entre préfixes

EXEMPLE

- Séquences : ACGGCTAT et ACTGTAT,
- Scores : match = 2, mismatch = -1 et indel= -2.

ALGORITHME

- $\text{Sim}(i ; j)$: score optimal entre $U(1;i)$ et $V(1;j)$
- Formule de récurrence :
 - $\text{Sim}(0; 0) = 0$
 - $\text{Sim}(0; j) = \text{Sim}(0; j - 1) + \text{Ins}(V(j))$
 - $\text{Sim}(i ; 0) = \text{Sim}(i - 1; 0) + \text{Del}(U(i))$
 - $\text{Sim}(i ; j) = \max \left\{ \begin{array}{l} \text{Sim}(i - 1; j - 1) + \text{Sub}(U(i); V(j)) \\ \text{Sim}(i - 1; j) + \text{Del}(U(i)) \\ \text{Sim}(i ; j - 1) + \text{Ins}(V(j)) \end{array} \right.$

ETAPE 1: CRÉATION D'UNE TABLE INDEXÉE PAR LES DEUX SÉQUENCES.

	-	A	C	G	G	C	T	A	T
-									
A									
C									
T									
G									
T									
A									
T									

- Case (i ; j) : score entre les i premières bases de ACGGCTAT et les j premières bases de ACTGTAT.

ETAPE 1: CRÉATION D'UNE TABLE INDEXÉE PAR LES DEUX SÉQUENCES.

	-	A	C	G	G	C	T	A	T
-	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2								
C	-4								
T	-6								
G	-8								
T	-10								
A	-12								
T	-14								

- Cas de base - initialisation

ETAPE 1: CRÉATION D'UNE TABLE INDEXÉE PAR LES DEUX SÉQUENCES.

	-	A	C	G	G	C	T	A	T
-	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	2	0	-2	-4	-6	-8	-10	-12
C	-4								
T	-6								
G	-8								
T	-10								
A	-12								
T	-14								

Max

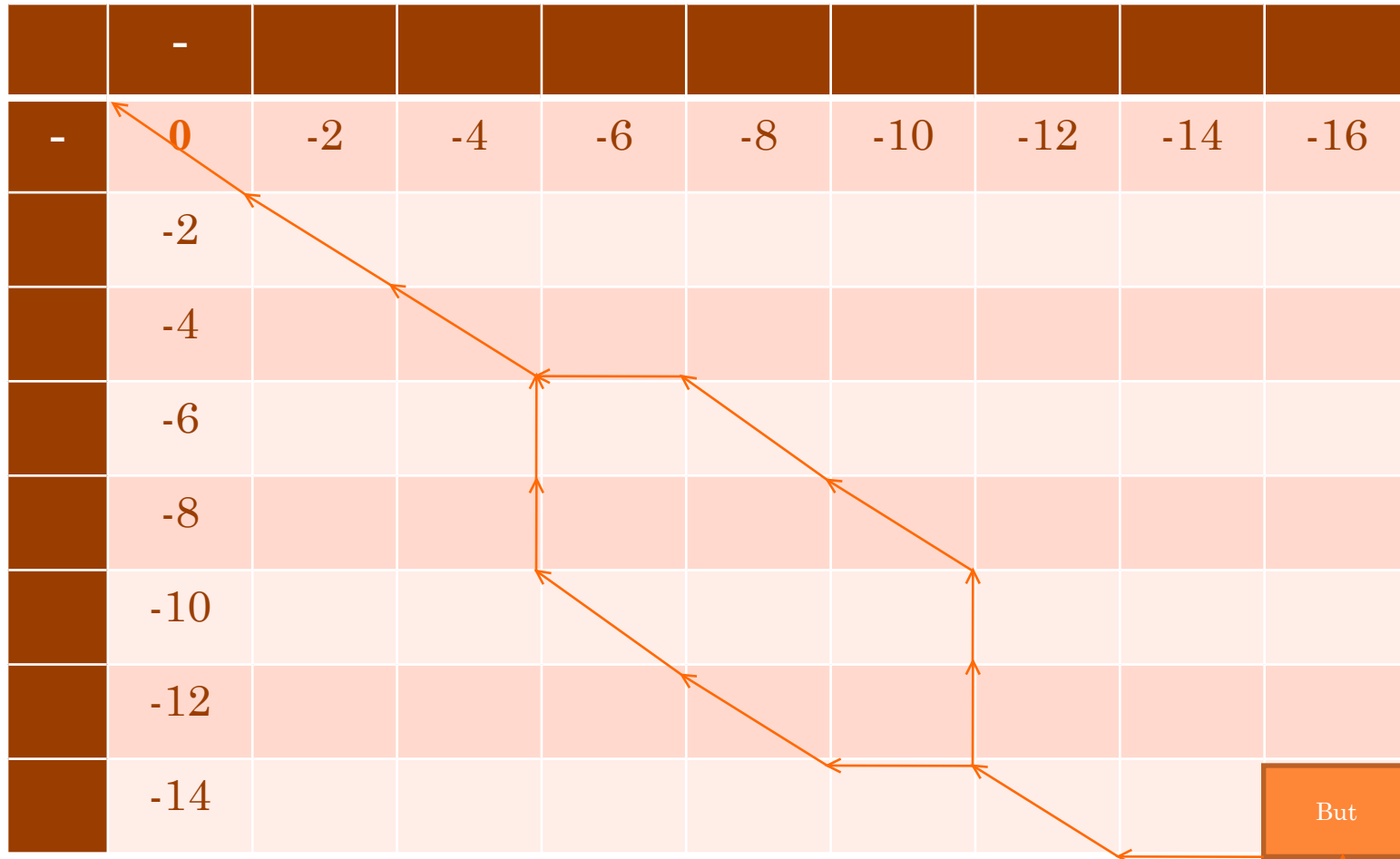
- $\text{Sim}(i - 1; j - 1) + \text{Sub}(U(i); V(j))$
- $\text{Sim}(i - 1; j) + \text{Del}(U(i))$
- $\text{Sim}(i; j - 1) + \text{Ins}(V(j))$

- Remplissage ligne à ligne
- On retient la direction fournissant le Max !

ETAPE 1: CRÉATION D'UNE TABLE INDEXÉE PAR LES DEUX SÉQUENCES.

- Complétez le reste de la table en retenant bien les directions.

ETAPE 2: RECHERCHE DU CHEMIN DE SCORES MAXIMAUX DANS LA MATRICE



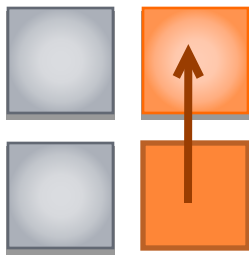
But

Meilleur score global 48

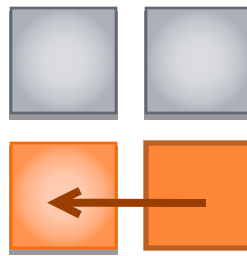
- Chemin inverse depuis le but
- Plusieurs chemins possibles ? Exemples ?

ETAPE 3 : CONSTRUCTION DE L'ALIGNEMENT

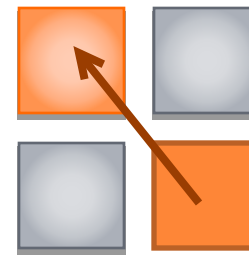
- Sur le chemin des scores maximaux, on regarde quelle est l'opération correspondante.



Insertion



Délétion



Substitution
Ou Identité

- Résultat

ACGGCTAT
?
ACTGTAT

ETAPE 1: CRÉATION D'UNE TABLE INDEXÉE PAR LES DEUX SÉQUENCES. (CORRECTION)

	-	A	C	G	G	C	T	A	T
-	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	2	0	-2	-4	-6	-8	-10	-12
C	-4	0	4	2	0	-2	-4	-6	-8
T	-6	-2	2	3	1	-1	0	-2	-4
G	-8	-4	0	4	5	3	1	-1	-3
T	-10	-6	-2	2	3	4	5	3	1
A	-12	-8	-4	0	1	2	3	7	5
T	-14	-10	-6	-2	-1	0	4	5	9

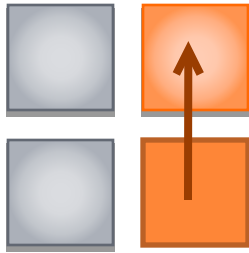
- Remplissage ligne à ligne

ETAPE 2: RECHERCHE DU CHEMIN DE SCORES MAXIMAUX DANS LA MATRICE (CORRECTION)

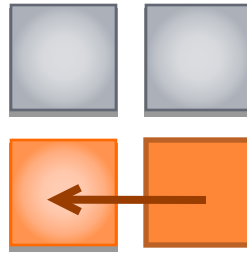
	-	A	C	G	G	C	T	A	T
-	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	2	0	-2	-4	-6	-8	-10	-12
C	-4	0	4	2	0	-2	-4	-6	-8
T	-6	-2	2	3	1	-1	0	-2	-4
G	-8	-4	0	4	5	3	1	-1	-3
T	-10	-6	-2	2	3	4	5	3	1
A	-12	-8	-4	0	1	2	3	7	5
T	-14	-10	-6	-2	-1	0	4	5	9

ETAPE 3 : CONSTRUCTION DE L'ALIGNEMENT (CORRECTION)

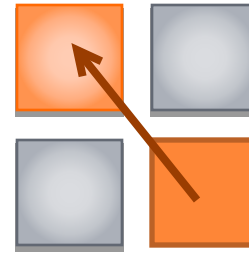
- Sur le chemin des scores maximaux, on regarde quelle est l'opération correspondante.



Insertion



Délétion



Substitution
Ou Identité

- Résultat

```
ACGGCTAT
|||||
ACTG-TAT
```

COMPLEXITÉ DE L'ALGORITHME

- Pour le calcul du score d'alignement : (étape 1)
 - $O(n \times m)$ en temps
 - $O(n \times m)$ en espace
- Pour la construction de l'alignement : (étapes 1, 2 et 3)
 - $O(n \times m)$ en temps et en espace

ENRICHISSEMENT DU MODÈLE: LES SCORES

○ Traitement des gaps

- Gap: succession de délétions ou d'insertions
- correspond à un seul évènement mutationnel (insertion ou disparition d'un bloc).

T C A G A C G A G T C T C A G A C G A G T C
 | | | | | | | | | | |
 T C G G A - G C - T C T C G G A - - G C T C

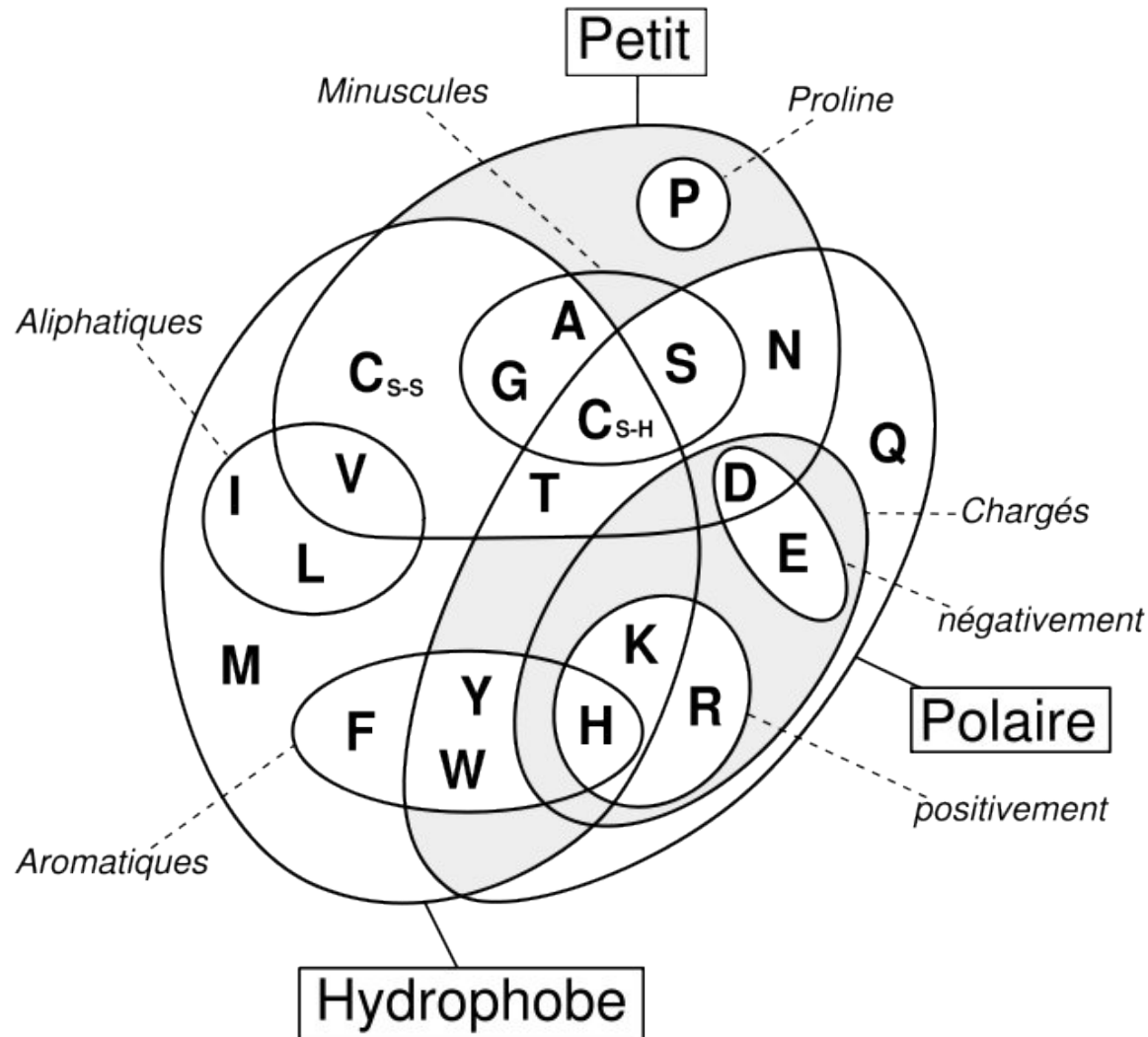
- nouvelles pénalités : pénalité d'ouverture de gap (exemple : -10) + pénalité d'extension de gap (exemple : -0.5)

- Score de substitutions

- ADN : identité, transition/transversion
- Protéines : matrice pour les 20 acides aminés

ALIGNEMENT DE SÉQUENCES PROTÉIQUES

- Matrices : PAM et BLOSUM



ALIGNEMENT GLOBAL

INFLUENCE DES SCORES

- On considère la matrice :

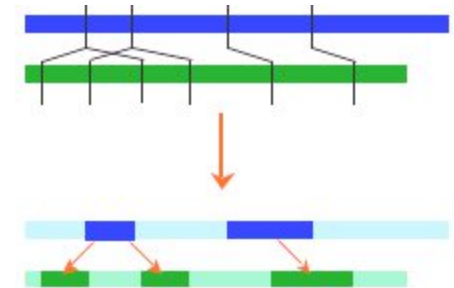
	A	C	G	T
A	2	-1	-1	-1
C	-1	2	-1	-1
G	-1	-1	2	-1
T	-1	-1	-1	2

- 1^{ère} hypothèse : pénalité de gap = **-2**
- 2^{ème} hypothèse : ouverture de gap = **-4**,
prolongation = **-1**
- Que se passe-t-il par exemple pour les séquences suivantes :
 - **AAGTCTCTAGAC**
 - **AATCTAC**

ALIGNEMENTS LOCAUX

○ Recherche du meilleur alignement entre 2 sous-séquences

- Partage d'un domaine commun
- Similarités entre séquences éloignées
 - Recherche de portions très similaires
- Plus grand score = Meilleur alignement local



○ Point de vue algorithmique

- Même principe
- Un alignement peut commencer et se terminer n'importe où dans les deux séquences

ALGORITHME

SMITH – WATERMAN (81)

- $\text{Sim}(i ; j)$: score optimal entre $U(1;i)$ et $V(1;j)$
- Formule de récurrence :
 - $\text{Sim}(0; j) = 0$
 - $\text{Sim}(i ; 0) = 0$
 - $\text{Sim}(i ; j) = \max \left\{ \begin{array}{l} \text{Sim}(i - 1; j - 1) + \text{Sub}(U(i); V(j)) \\ \text{Sim}(i - 1; j) + \text{Del}(U(i)) \\ \text{Sim}(i ; j - 1) + \text{Ins}(V(j)) \\ 0 \end{array} \right.$

ALGORITHME

SMITH – WATERMAN (81)

- On calcule alors la table T de programmation dynamique associée
- On recherche la case contenant le maximum
 - Plus nécessairement $T[n,m]$
- On remonte l'alignement jusqu'à ce qu'on trouve un 0
- On construit l'alignement suivant les mêmes règles que pour l'alignement global

ALIGNEMENT LOCAL

EXEMPLE

- Séquences : ACGTCTAT et ACTGTAT,

	A	C	G	T
A	2	-1	-1	-1
C	-1	2	-1	-1
G	-1	-1	2	-1
T	-1	-1	-1	2

- Scores : et indel= -2.

- Quels sont les meilleurs alignements locaux

ALIGNEMENT LOCAL

MATRICE DE PROGRAMMATION DYNAMIQUE

	-	A	C	G	T	C	T	A	T
-	0	0	0	0	0	0	0	0	0
A	0	2	0	0	0	0	0	2	0
C	0	0	4	2	0	2	0	0	1
T	0	0	2	3	1	0	4	2	2
G	0	0	0	4	2	0	2	3	1
T	0	0	0	2	6	4	2	1	5
A	0	2	0	0	4	5	3	4	3
T	0	0	1	0	2	3	7	5	6

ALIGNEMENT LOCAL

EXEMPLE

- Séquences : ACGTCTAT et ACTGTAT,
- Meilleur alignement local : valeur 7, longueur 7

A	C	-	G	T	C	T	A	T
A	C	T	G	T	A	T		

OUTILS DISPONIBLES

○ BLAST (90)

- Approche heuristique
- Trouver des zones très courtes d'identités ou de scores élevés
- Étendre ces zones

○ FASTA (88)

- Recherche d'appariements exacts de mots de taille fixée
- Extension à des régions sans gap et de score maximal
- Extension à des régions avec gap et de score maximal
- Recherche restreinte à ces zones par programmation dynamique

○ ...